



Fakulteti i Shkencave Kompjuterike dhe Inxhinierisë

Tema: Parashikimi dhe grupimi i performancës akademike të studentëve duke përdorur metoda të ndryshme të mësimit makinerik

Lënda: Modelet e Mësimit Makinor

Mentor: PhD(c) Greta Ahma

Student: Fjolla Nebihi

Prishtinë, Qershor 2026

Table of Content

1. Hyrje	3
2. Dataset	4
3. Metodologjia	5
4. Rezultatet	11
5. Diskutimi	19
6. Përfundimi	21
7. Referencat	23

Table of Figures

<i>Figure 1</i> .Confusion Matrix per algoritmin KNN	11
<i>Figure 2</i> . Confusion Matrix per Logistic Regression	12
<i>Figure 3</i> .Confusion Matrix for Neural Network	14
<i>Figure 4</i> . Fully Connected Neural Network	14
<i>Figure 5</i> . Confusion Matrix for Random Forest	15
<i>Figure 6</i> . K-Means Clustering	17
<i>Figure 7</i> . Hierarchical Clustering	18
<i>Figure 8</i> .Krahasimi I Klasifikatoreve	22

1. Hyrje

Qëllimi i këtij projekti është analiza e performancës akademike të nxënësve duke përdorur metoda të ndryshme të të mësuarit makinerik, si për klasifikim ashtu edhe për klasterizim. Fokusi është të kuptohet se si faktorë të ndryshëm akademikë dhe sjellorë (p.sh. koha e studimit, mungesat, aktivitetet jashtëshkollore, mbështetja prindërore) lidhen me notën përfundimtare të studentëve dhe se sa mirë mund të parashikohen këto nota duke përdorur modele të ndryshme.

Nga ana klasifikuese, projekti implementon katër algoritme të ndryshëm: K-Nearest Neighbors (KNN), Regresion Logjistik, një neural network shumë-shtresor (MLP) dhe Random Forest. Secili model trajnohet mbi të njëjtin pipeline parapërpunimi, i kalon fazës së përzgjedhjes së veçorive dhe i nënshtrohet rregullimit të hiperparametrave përmes GridSearchCV me kross-validim 5-fold. Qëllimi është jo vetëm të gjendet modeli me performancën më të mirë, por edhe të analizohet ndikimi i përzgjedhjes së veçorive dhe i konfigurimeve të ndryshme të modeleve në rezultatet përfundimtare.

Përveç klasifikimit, projekti përfshin edhe klasterizim (K-Means dhe klasterizim hierarkik) me qëllim të eksplorimit të strukturës së brendshme të të dhënave pa përdorur etiketat gjatë trajnimit. Rezultatet e klasterizimit krahasohen më pas me etiketat e vërteta nëpërmjet metrikave si Adjusted Rand Index (ARI) dhe Normalized Mutual Information (NMI), duke dhënë një perspektivë shtesë mbi mënyrën se si grupohen natyrshëm studentët sipas performancës së tyre.

2. Dataset

Të dhënat e përdorura në këtë projekt vijnë nga “Students Academic Performance Evaluation Dataset”, i cili përmban të dhëna reale mbi performancën akademike të 2392 studentëve. Çdo rresht përfaqëson një student, ndërsa dataset- i përfshin gjithsej 15 kolona, duke kombinuar veçori demografike, akademike dhe sjellore.

Veçoritë kryesore janë:

1. ***StudentID*** – identifikues unik për secilin student (përdoret vetëm për identifikim dhe hiqet si veçori gjatë modelimit).
2. ***Age*** – mosha e studentit.
3. ***Gender*** – gjinia e kodifikuar në format numerik.
4. ***Ethnicity*** – etnia e studentit, e përfaqësuar me kode të ndryshme.
5. ***ParentalEducation*** – niveli i edukimit të prindërve.
6. ***StudyTimeWeekly*** – koha mesatare javore e studimit (në orë).
7. ***Absences*** – numri i mungesave gjatë vitit.
8. ***Tutoring*** – pjesëmarrja në tutorim (po/jo, e kodifikuar numerikisht).
9. ***ParentalSupport*** – niveli i mbështetjes prindërore.
10. ***Extracurricular, Sports, Music, Volunteering*** – pjesëmarrja në aktivitete jashtëshkollore, sport, muzikë dhe vullnetarizëm.
11. ***GPA*** – nota mesatare (Grade Point Average).
12. ***GradeClass*** – etiketa e klasës së notës (nga 0 në 4), e cila shërben si variabël target për klasifikim.

Dataset- i nuk përmban vlera që mungojnë; megjithatë, pipeline- i i parapërpunimit është ndërtuar duke supozuar një skenar përgjithësues (me imputim median për numeriket dhe most_frequent për kategoriket), në mënyrë që zgjidhja të jetë e qëndrueshme edhe në kontekste të tjera. Shpërndarja e klasave në GradeClass është e pabalancuar: klasa 4 ka numrin më të madh të rasteve, ndërsa klasat e ulëta (p.sh. 0 dhe 1) kanë shumë më pak vëzhgime. Kjo e bën zgjedhjen e metrikave të bazuara në F1-macro veçanërisht të përshtatshme për vlerësim.

3. Metodologjia

3.1 Mjedisi i zhvillimit

Projekti është realizuar duke përdorur gjuhën e programimit *Python 3*, në një mjedis të izoluar virtual (virtual environment), në mënyrë që varësitë e bibliotekave të jenë të kontrolluara dhe të mos ndërthuren me instalimet globale të sistemit. Krijimi i mjedisit virtual është bërë me komandën *python -m venv*, ndërsa aktivizimi i tij është kryer përpara instalimit të paketave dhe ekzekutimit të notebook-ëve. Kjo qasje siguron që të gjitha eksperimentet të mund të riprodhohen lehtë edhe në kompjuterë të tjerë, mjafton të instalohet e njëjta listë paketash.

Pjesa kryesore e zhvillimit është bërë në **Jupyter Notebook / JupyterLab**. Çdo algoritëm klasifikimi apo klasterizimi është implementuar në një notebook të veçantë, duke lejuar ndarjen e pastër të kodeve, vizualizimeve dhe komenteve shpjeguese. Kjo strukturë bën të mundur që i gjithë procesi – nga ngarkimi i datasetit, parapërpunimi, ndërtimi i pipeline-ëve, trajnimi i modeleve, vlerësimi i metrikave dhe deri te ruajtja e rezultateve – të dokumentohet hap-pas-hapi në një formë interaktive.

Zhvillimi bazohet në ekosistemin klasik të Python për data science: pandas dhe numpy për menaxhimin e të dhënave, scikit-learn për implementimin e algoritmeve të mësimi makinerik dhe pipeline-ëve, ndërsa matplotlib dhe seaborn për vizualizime grafike. Projekti është organizuar në mënyrë hierarkike nëpër dosje (**data/**, **notebooks/**, **results/**, **visualizations/**, **documentation/**), në mënyrë që të jetë i lehtë orientimi dhe menaxhimi i skedarëve për çdo fazë të punës.

3.2 Parapërpunimi i të dhënave

Parapërpunimi i të dhënave është një hap kritik, pasi cilësia e inputit ndikon drejtpërdrejt në performancën e modeleve. Fillimisht, dataset-i ngarkohet nga skedari CSV me `pandas.read_csv`, dhe kryhen kontrolle bazë si dimensionet e të dhënave, tipet e kolonave dhe statistikat përmbledhëse. Në këtë fazë identifikohet

kolona GradeClass si variabël target, ndërsa kolonat jo-informative për parashikimin (si StudentID) hiqen nga matrica e veçorive X.

Pasi ndarja X/Y është bërë, të dhënat ndahen në set trajnimi dhe testimi me raport 80/20, duke përdorur stratifikim sipas GradeClass për të ruajtur të njëjtën shpërndarje të klasave në të dy subsetet. Në vijim, përcaktohen dy grupe veçorish: numerike dhe kategorike. Në këtë dataset shumica e veçorive janë numerike, por metodologjia është ndërtuar në mënyrë të përgjithshme, duke përdorur një ColumnTransformer që mund të trajtojë edhe veçori kategorike nëse shtohen në të ardhmen.

Për veçoritë numerike, parapërpunimi përbëhet nga dy hapa: (1) imputimi i vlerave të mungura me median për të reduktuar ndikimin e outlier-ëve, dhe (2) standardizimi me StandardScaler për t'i sjellë të gjitha veçoritë në shkallë të krahasueshme (mesatare 0, devijim standard 1). Kjo është veçanërisht e rëndësishme për algoritme si KNN dhe rrjetet nervore, të cilat janë shumë të ndjeshme ndaj shkallës së veçorive.

Për veçoritë kategorike (nëse ekzistojnë), përdoret imputimi me vlerën më të shpeshtë dhe OneHotEncoder(handle_unknown="ignore"), i cili krijon kolonat binare për secilën kategori dhe trajton korrekt edhe vlerat e papara më herët.

E gjithë kjo logjikë parapërpunimi integrohet brenda një pipeline- i së bashku me përzgjedhjen e veçorive dhe klasifikuesin. Kjo siguron që e njëjta seri transformimesh të aplikohet njësoj si në trajnimin me cross validation, ashtu edhe në vlerësimin final në setin e testit, duke shmangur gabimet e mundshme si “data leakage”.

3.3 Përzgjedhja e veçorive

Përzgjedhja e veçorive synon të identifikojë nënshtresën më informuese të veçorive që japin kontributin kryesor në parashikimin e **GradeClass**, duke reduktuar njëkohësisht zhurmën dhe kompleksitetin e modelit. Në këtë projekt është përdorur qasja filter me SelectKBest dhe funksionin e vlerësimit f_classif, i cili bazohet në ANOVA F-test për të matur rëndësinë statistike të secilës veçori në dallimin mes klasave.

Metodologjia nuk fikson paraprakisht një numër të veçorive, por e trajton k- në si hiperparametër që duhet gjetur. Për të njëjtin pipeline parapërpunimi + SelectKBest + klasifikues, testohen disa vlera të ndryshme për k (3, 5, 7, 10 dhe “të gjitha veçoritë”). Kjo bëhet në kuadër të GridSearchCV, që do të thotë se për çdo kombinim të

hiperparametrave të klasifikuesit, testohen edhe vlera të ndryshme të k -së. Në fund, si më e mirë zgjidhet ajo vlerë e k -së që jep F1-macro më të lartë në kross-validim. Në këtë dataset, për të gjithë klasifikuesit, rezulton se $k=3$ është vlera optimale. Kjo do të thotë që vetëm tre veçori (nga dhjetëra kolonat fillestare, pas përfshirjes edhe të one-hot encoding-ut) mjaftojnë për të arritur performancë shumë të lartë. Ky rezultat tregon se informacioni kryesor për dallimin e klasave të notës është i përqendruar në një numër të vogël veçorish, ndërsa të tjerat janë ose të tepërta, ose të lidhura fort me këto të parat. Kjo e bën modelin më të lehtë për t'u interpretuar dhe redukton rrezikun e overfitting.

3.4 Klasifikuesit e përdorur

Në projekt janë përdorur katër lloje klasifikuesish, të zgjedhur për të përfaqësuar qasje të ndryshme: algoritme të bazuara në distancë, modele lineare, rrjete nervore dhe ansamble pemësh.

K-Nearest Neighbors (KNN)

KNN është një algoritëm i thjeshtë, por intuitiv: për një pikë të re, ai gjen k fqinjët më të afërt në hapësirën e veçorive dhe ia jep klasën më të shpeshtë mes tyre. Ai është shumë i ndjeshëm ndaj shkallës së veçorive, prandaj standardizimi para aplikimit është i domosdoshëm. Në këtë projekt, KNN përdoret si model bazë, me parametra si `n_neighbors`, `weights` dhe `metric` që përcaktojnë sa fqinjë merren parasysh, si peshohen ata dhe me cilën metrikë distance matet afërsia.

Regresioni Logjistik

Regresioni logjistik është një model linear probabilistik që përdor funksionin logit për të llogaritur probabilitetet e secilës klasë. Për klasifikimin multi-klasor, zbatohet qasja “one- vs-rest” ose **softmax**, në varësi të implementimit. Ky model përdoret shpesh si baseline për shkak të thjeshtësisë dhe interpretueshmërisë, pasi koeficientët e tij mund të lidhen drejtpërdrejt me ndikimin e veçorive. Në këtë projekt testohet ndikimi i parametrave të rregullimit C , i llojit të penalizimit (p.sh. L2, elasticnet) dhe solverëve të ndryshëm që përshtaten me natyrën e problemit.

Multi Layer Neural Network

MLP është një rrjet nervor “feedforward” me një ose disa shtresa të fshehta, ku çdo neuroni aplikon një transformim jolinear mbi kombinimin linear të hyrjeve. Këtu, MLP përfaqëson qasjen e “deep learning” për të njëjtin dataset, duke mundësuar kapjen e marrëdhënieve jolineare ndërmjet veçorive. Testohen dy arkitektura – një me dy shtresa të fshehta dhe një tjetër me tre shtresa – si dhe dy lloje funksionesh aktivizimi (ReLU dhe Tanh). Parametrat si alpha (rregullimi L2) dhe learning_rate_init kontrollojnë nivelin e rregullimit dhe shpejtësinë e mësimi. Përdorimi i early_stopping bën që trajnimi të ndalet automatikisht kur performanca në setin e validimit nuk përmirësohet më, duke shmangur overfitting-un.

Random Forest

Random Forest është një algoritëm ansambël që ndërton shumë pemë vendimmarrjeje mbi mostra të ndryshme të të dhënave dhe të veçorive, dhe pastaj kombinon parashikimet e tyre (p.sh. me votim shumice). Kjo qasje rrit stabilitetin dhe ul variancën në krahasim me një pemë të vetme, duke e bërë modelin shumë të fuqishëm dhe relativisht rezistent ndaj zhurmës. Parametrat kryesorë janë numri i pemëve (n_estimators), thellësia maksimale e tyre (max_depth) dhe numri minimal i mostrave për ndarje (min_samples_split). Random Forest është i aftë të kapë ndërveprime komplekse dhe jolineare, pa pasur nevojë për shumë parapërpunim shtesë, ndaj shpesh rezulton model shumë konkurrues në probleme të tilla.

3.5 Hyperparameter tuning dhe validimi

Çdo klasifikues ka një grup parametrash të brendshëm (hiperparametra) që nuk mësohen në mënyrë automatike nga të dhënat, por duhen zgjedhur nga përdoruesi. Zgjedhja manuale mund të jetë subjektive dhe jo optimale, prandaj në këtë projekt përdoret **GridSearchCV**, e cila kryen kërkim sistematik nëpër një grid vlerash për çdo hiperparametër.

Për secilin model ndërtohet një grid i veçantë që përfshin, për shembull:

Për KNN: vlera të ndryshme për `n_neighbors`, lloje të ndryshme `weights` dhe metrika të distancës.

Për Regresionin Logjistik: vlera të ndryshme të `C`, kombinime të `penalty` dhe `solver`.

Për MLP: kombinime të ndryshme të madhësisë së shtresave të fshehta, funksione aktivizimi, `alpha` dhe `learning_rate_init`.

Për Random Forest: vlera të ndryshme për `n_estimators`, `max_depth` dhe `min_samples_split`.

GridSearchCV përdor kross- validim 5- fold, domethënë seti i trajnimit ndahet në 5 pjesë, dhe për çdo kombinim hiperparametrash modeli trajnohet 5 herë, duke përdorur 4 pjesë për trajnim dhe 1 për validim, rotacioni vazhdon derisa çdo pjesë të ketë shërbyer si validim. Metrika e përdorur për të vlerësuar secilin konfigurim është F1- macro, e cila llogarit F1- score për secilën klasë dhe pastaj merr mesataren e tyre të paveshuar (macro average). Kjo bën që edhe klasat me pak shembuj të kenë peshë të barabartë në procesin e zgjedhjes së hiperparametrave.

Pasi GridSearchCV përfundon, zgjidhet kombinimi me F1- macro më të lartë mesatare në kross- validim, dhe ky model ritrajtohet (nëse është e nevojshme) në të gjithë setin e trajnimit. Vetëm më pas modeli vlerësohet në setin e testit, që deri në atë moment ka mbetur i paprekur, duke siguruar një vlerësim sa më të paanshëm të performancës përfundimtare.

3.6 Klasterizimi

Përveç klasifikimit të mbikëqyrur, projekti përfshin edhe detyrën e klasterizimit, ku nuk përdoren etiketa gjatë trajnimit, por vetëm pas tij, për **vlerësim**. Në këtë fazë, kolona GradeClass hiqet nga matrica e veçorive dhe ruhet veçmas për të llogaritur më vonë metrikat që krahasojnë klasterët me të vërtetën (ARI, NMI). Për pjesën e klasterizimit përdoret një pipeline parapërpunimi i ngjashëm me atë të klasifikimit, ku veçoritë numerike standardizohen përpara aplikimit të algoritmeve.

Dy qasje kryesore përdoren:

1. K- Means Clustering

K- Means synon të ndajë të dhënat në `k` klasterë duke minimizuar distancën mes pikave dhe qendrave (centroidave) të klasterëve. Problemi është që `k` nuk dihet

paraprakisht, prandaj testohen vlera të ndryshme (p.sh. 3, 4, 5, 6). Për secilin k llogaritet Silhouette Score, i cili mat se sa mirë është e izoluar çdo pikë nga klasterët e tjerë (vlerat afër 1 janë më të mira). Përveç kësaj, për çdo konfigurim llogariten edhe Adjusted Rand Index (ARI) dhe Normalized Mutual Information (NMI) duke krahasuar etiketat e klasterëve me GradeClass. Këto metrika tregojnë sa mirë përputhen klasterët e gjetur me klasat e vërteta, edhe pse algoritmi nuk i ka parë ato gjatë trajnimit.

2. Klasterizimi hierarkik (agglomerativ)

Në këtë qasje, çdo pikë fillimisht shihet si klaster më vete, dhe më pas klasterët bashkohen gradualisht sipas një kriteri linkage (p.sh. ward, average, complete), derisa të arrihet numri i dëshiruar i klasterëve. Ashtu si në K- Means, testohen disa vlera për numrin e klasterëve (4, 5, 6) dhe kombinime të ndryshme me lloje linkage- sh. Për çdo konfigurim llogaritet Silhouette Score për të vlerësuar ndarjen e klasterëve, dhe gjithashtu ARI/NMI për të parë sa përputhen klasterët me GradeClass. Rezultatet e vlerësimit regjistrohen në skedarë të veçantë dhe vizualizohen nëpërmjet një PCA 2D, ku çdo klaster paraqitet me ngjyrë të ndryshme.

Këto dy metoda të klasterizimit japin një perspektivë alternative, “unsupervised”, mbi strukturën e të dhënave: ato tregojnë se në çfarë mënyrash studentët grupohen në bazë të karakteristikave të tyre, pavarësisht se si janë kategorizuar formalisht nga sistemi i notimit.

4. Rezultatet

4.1 Rezultatet e klasifikimit

Për secilin klasifikues janë llogaritur metrikat kryesore të performancës (Accuracy, Precision- macro, Recall- macro dhe F1- macro) në setin e testit, si dhe është ruajtur një matricë konfuzioni për analizën e detajuar të gabimeve.

KNN (k- Nearest Neighbors)

Modelet KNN u trajnuan brenda pipeline- it me parapërpunim, SelectKBest dhe GridSearchCV. Konfigurimi më i mirë doli me $n_neighbors=11$, $weights='distance'$, $metric='manhattan'$ dhe $k=3$. Ky model arriti rreth 0.85 saktësi në test dhe një F1- macro rreth 0.75, që tregon performancë të mirë, por më të ulët krahasuar me modelet më të avancuara.

Figura 1 tregon se klasat me notë të lartë (sidomos GradeClass=4) parashikohen më mirë, ndërsa klasa 0 ka më shumë gabime – shpesh ngatërrohet me klasën 1 apo 2.

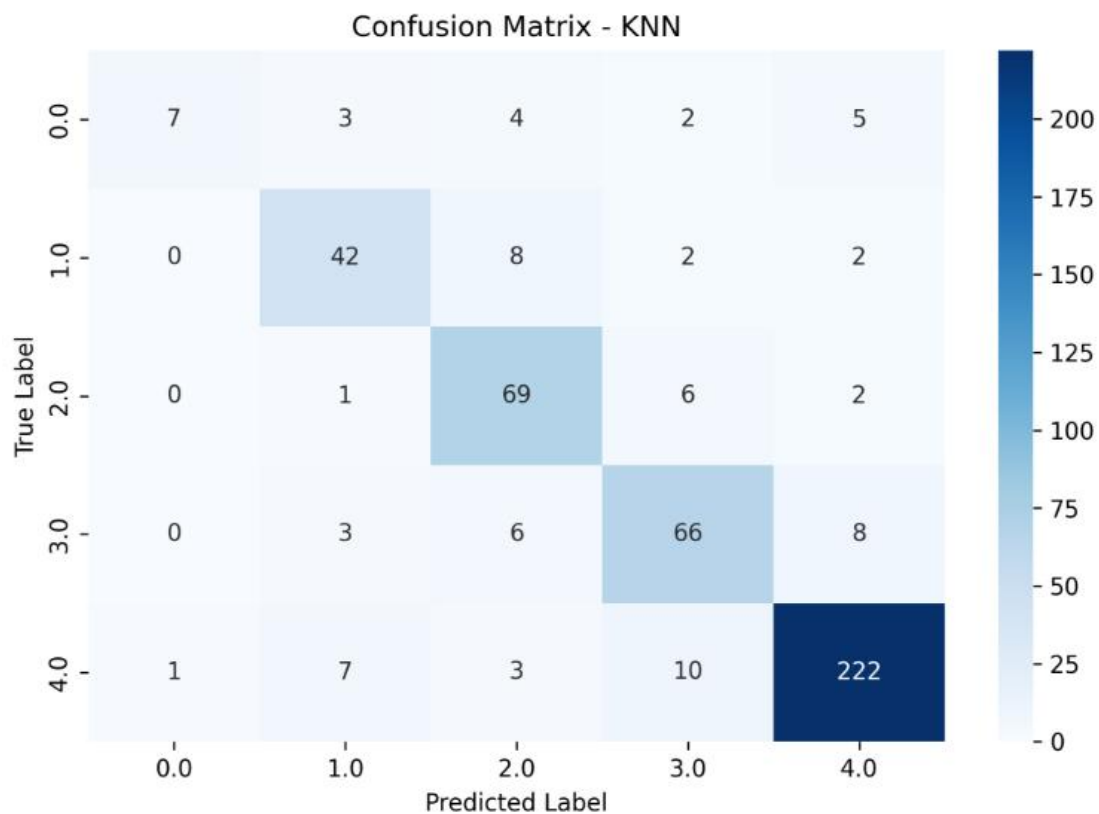


Figure 1. Confusion Matrix per algoritmin KNN

Regresioni Logjistik

Regresioni logjistik me rregullim L2, C=10, solver='lbfgs' dhe k=3 nga SelectKBest jep F1- macro rreth 0.67 në kross- validim, dhe metrika pak më të ulëta në test se KNN.

Figura 2 tregon që modeli ka tendencë të thjeshtojë strukturën dhe shpesh preferon klasat më të mëdha, duke lënë më pak të sakta klasat me pak shembuj.



Figure 2. Confusion Matrix per Logistic Regression

Neural Network (MLP)

Rrjeti nervor MLP është eksperimenti kryesor “deep learning” në projekt. Dy arkitektura u testuan: (64, 32) dhe (128, 64, 32), me aktivizimet ReLU dhe Tanh, dy vlera për alpha dhe dy vlera për learning_rate_init.

GridSearchCV me 5- fold CV gjeti konfigurimin optimal:

Arkitektura: (128, 64, 32)
Aktivizim: ReLU
alpha = 0.001
learning_rate_init = 0.01
k = 3 (SelectKBest)
Me këtë konfigurim, në test u arritën:
Accuracy ≈ 0.91
Precision- macro ≈ 0.91
Recall- macro ≈ 0.81
F1- macro ≈ 0.84

Raporti i klasifikimit tregon se klasat 2, 3 dhe 4 kanë precision dhe recall të larta, ndërsa klasa 0 mbetet më e vështira (recall më i ulët, pra shumë nga rastet e saj parashikohen si klasa fqinje).

Matrica e konfuzionit për MLP (figura 3) e bën këtë qartë duke shfaqur një diagonalë kryesore të fortë, me ndryshime të vogla në klasat e ulëta.

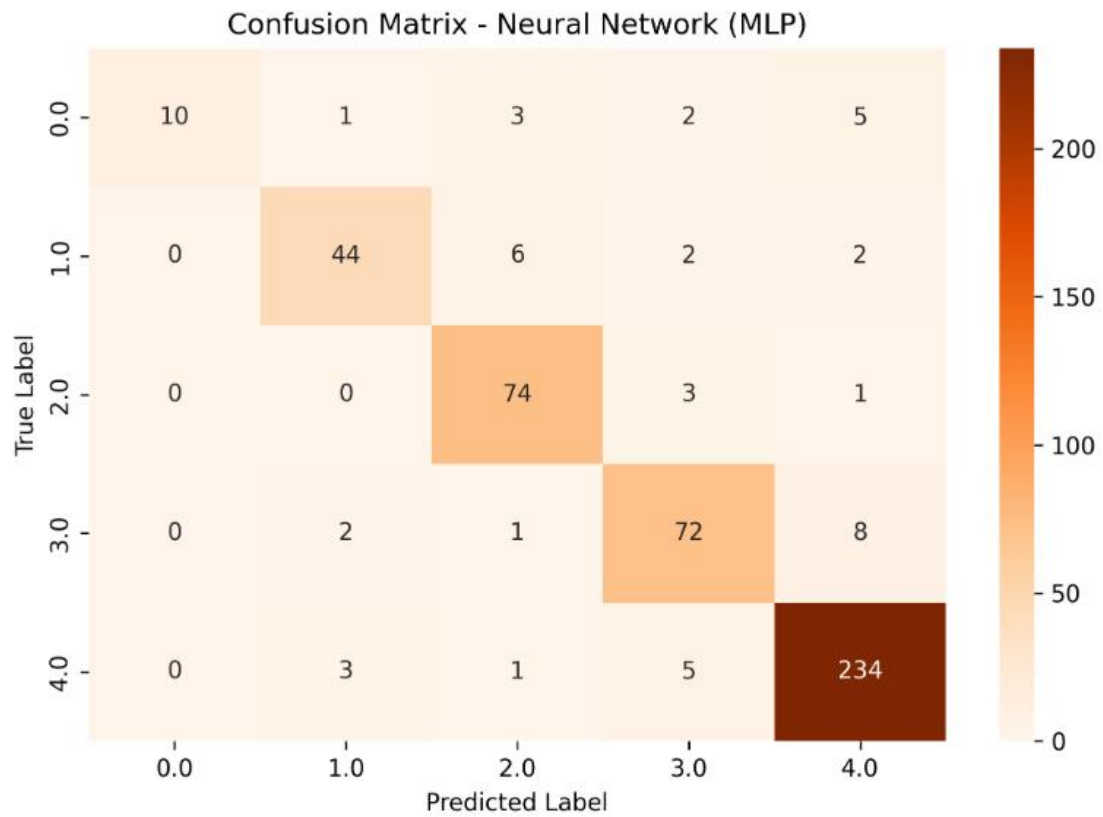


Figure 3. Confusion Matrix for Neural Network

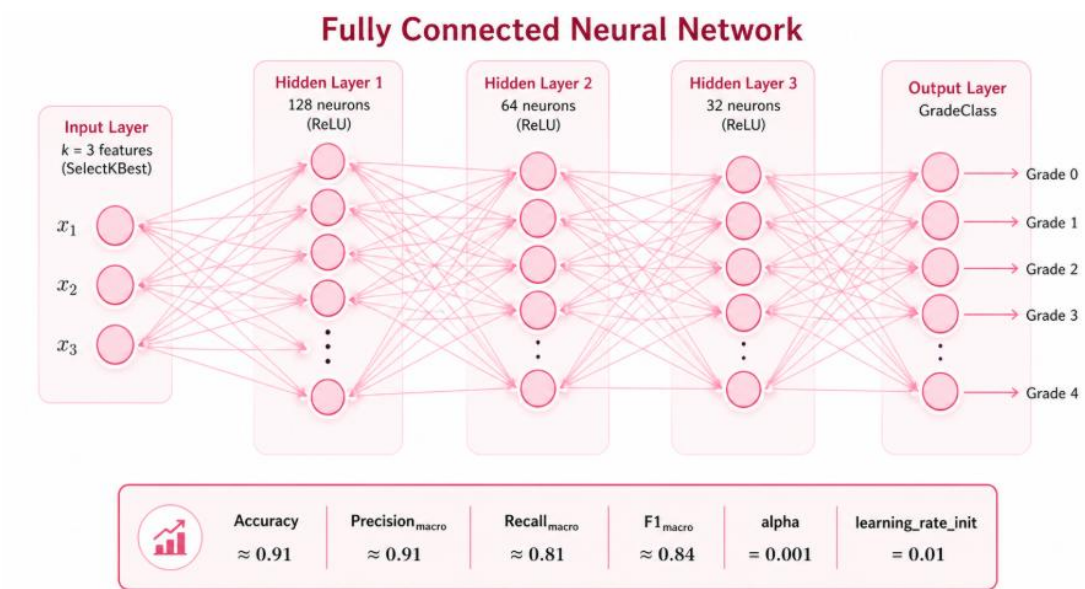


Figure 4. Fully Connected Neural Network

Random Forest

Random Forest u trajnuan me pipeline të njëjtë parapërpunimi dhe SelectKBest, duke testuar `n_estimators` (50, 100), `max_depth` (None, 10, 20) dhe `min_samples_split` (2, 5).

Kombinimi optimal doli: `n_estimators=50`, `max_depth=None`, `min_samples_split=5`, me `k=3`.

Ky model arriti rezultate edhe më të larta në test:

Accuracy ≈ 0.92

Precision- macro, Recall- macro dhe F1- macro shumë pranë vlerave të MLP, në disa raste pak më të larta.

Matrica e konfuzionit (figura 5) tregon diagonal shumë të pastër, me numër minimal gabimesh për klasat e mëdha dhe një trajtim gjithashtu të mirë për klasat më të vogla, krahasuar me modelet e tjera.

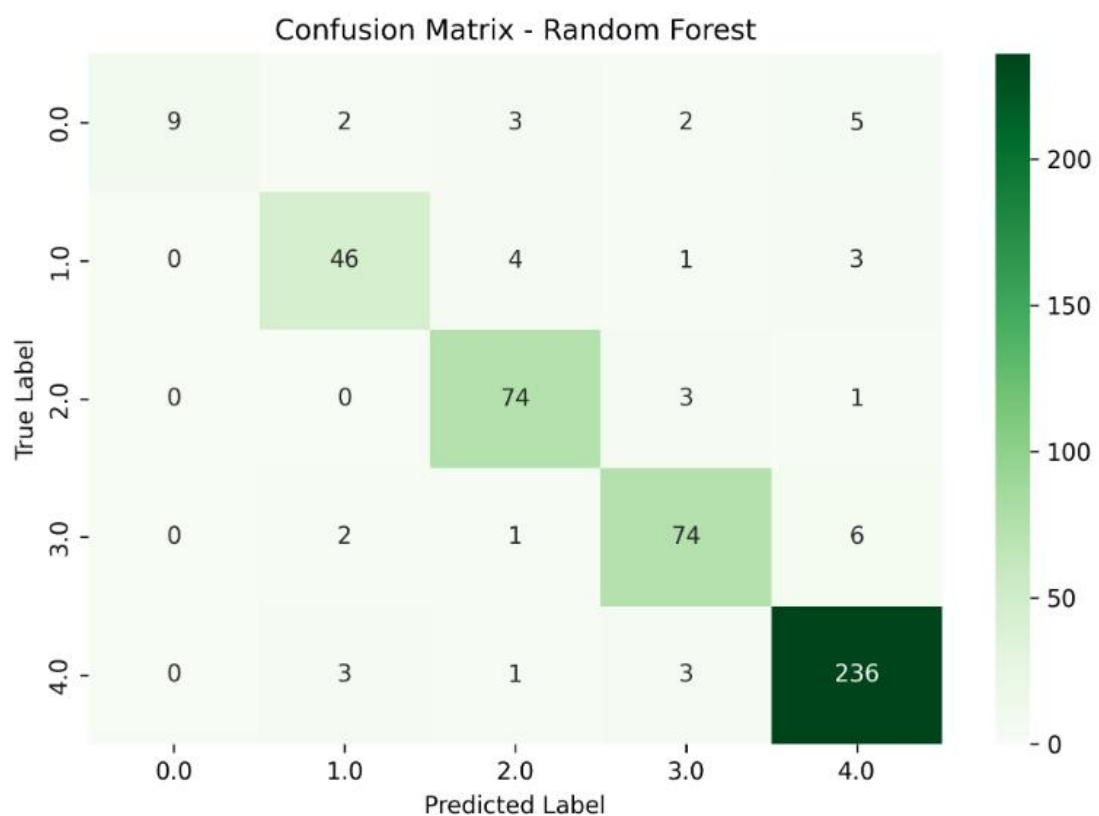


Figure 5. Confusion Matrix for Random Forest

Grafiku i krahasimit të klasifikuesve

Të gjitha rezultatet e modeleve (CV F1- macro dhe metrikat në test) janë bashkuar në `results/all_classifiers_comparison.csv` dhe janë vizualizuar me një grafik me shtylla

(visualizations/classifiers_comparison.png).

Në këtë figurë, për secilin model shfaqen Test Accuracy, Test Precision- macro, Test Recall- macro dhe Test F1- macro, duke e bërë të qartë renditjen: Random Forest dhe MLP dalin më lart, të ndjekur nga KNN dhe në fund Regresioni Logjistik.

4.2 Rezultatet e klasterizimit

Pjesa e klasterizimit synon të analizojë strukturën e të dhënave pa përdorur etiketat gjatë trajnimit dhe më pas të shohë sa mirë përputhen klasterët me klasat e vërteta (GradeClass).

1. K- Means

Në notebook- un kmeans_01.ipynb janë testuar vlerat $k = 3, 4, 5, 6$. Për secilin k janë llogaritur:

Silhouette Score – për të matur kohezionin dhe ndarjen e klasterëve.

Adjusted Rand Index (ARI) – sa mirë përputhen klasterët me etiketat.

Normalized Mutual Information (NMI) – masë informacioni e përbashkët mes klasave dhe klasterëve.

Rezultatet janë ruajtur në results/kmeans_results.csv dhe tregon se një vlerë e ndërmjetme e k (p.sh. $k=4$) ofron Silhouette Score më të mirë, duke sugjeruar një ndarje të arsyeshme të të dhënave në katër grupe.

Figura 6 visualizations/kmeans_clusters_pca.png paraqet klasterët në 2D pas reduktimit të dimensionit me PCA. Pikët janë ngjyrosur sipas klasterit dhe shihet se disa klasterë janë mirë të ndarë, ndërsa të tjerë mbivendosen, gjë që përshtatet me vlerat e moderuara të ARI dhe NMI.

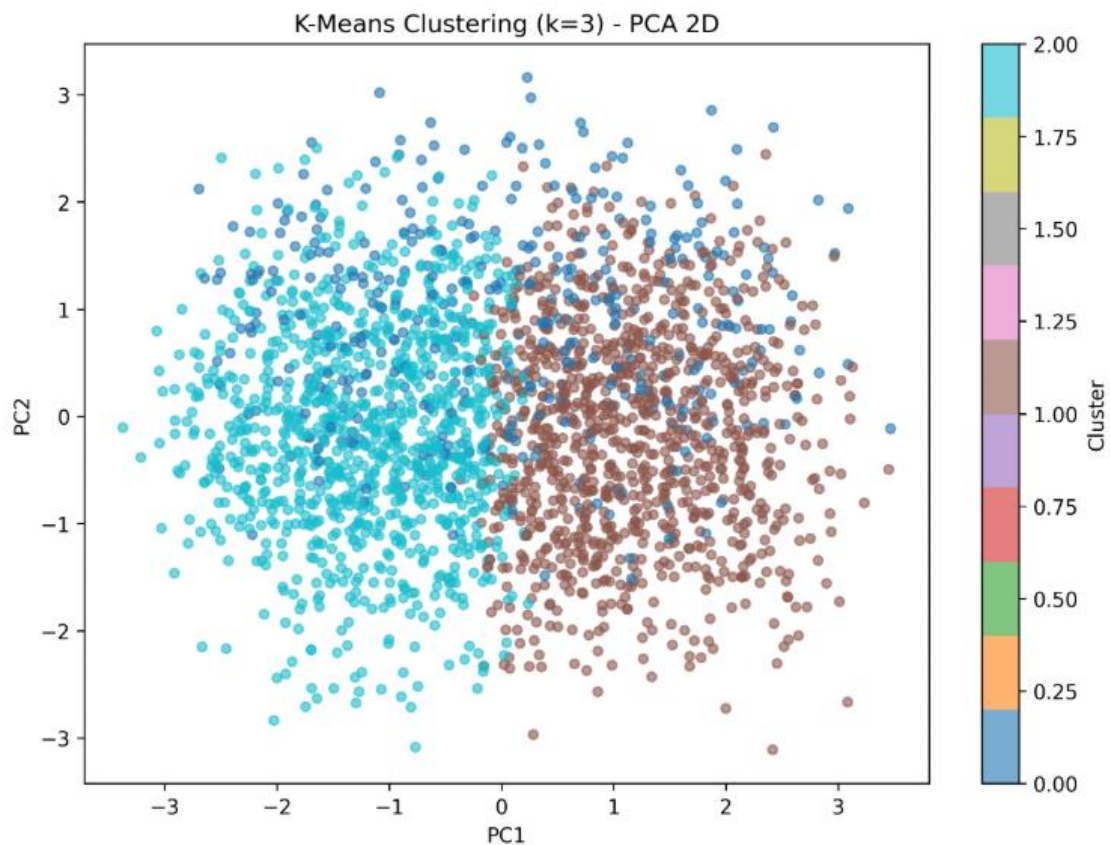


Figure 6. K-Means Clustering

2. Klasterizimi hierarkik

Në hierarchical_02.ipynb është testuar klasterizimi hierarkik me numra klasterësh 4, 5, 6 dhe linkage të ndryshme (ward, average, complete).

Sërish, për secilin konfigurim llogariten Silhouette, ARI dhe NMI. Rezultatet ruhen në results/hierarchical_results.csv.

Në përgjithësi, linkage ward rezulton më i përshtatshëm për këtë dataset, duke dhënë vlera pak më të larta të Silhouette Score. Figura 7 paraqet klasterët hierarkikë në 2D (PCA), ku shihet në cilat zona të hapësirës klasterët janë kompakt dhe ku ka ndërthurje mes tyre.

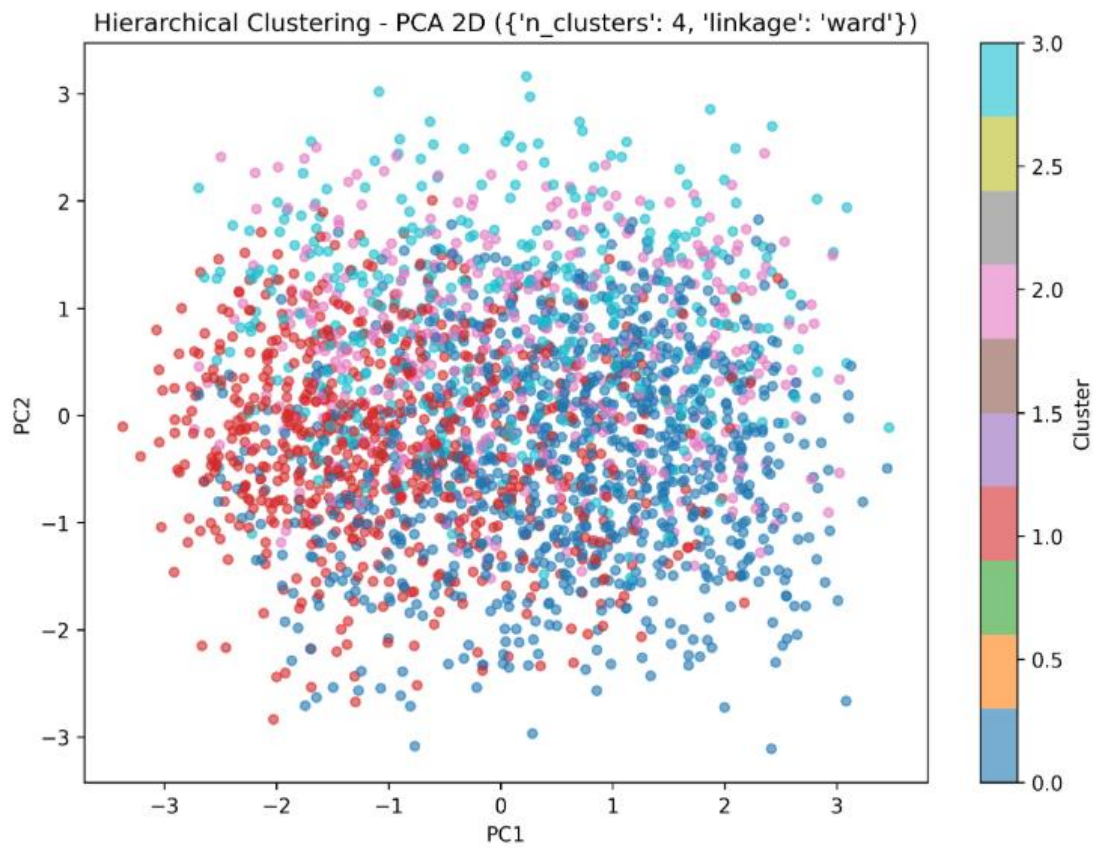


Figure 7. Hierarchical Clustering

5. Diskutimi

Rezultatet e klasifikimit tregojnë qartë se zgjedhja e modelit ka ndikim të madh në performancë. KNN dhe Regresioni Logjistik shërbejnë si baselines të mira: KNN përfiton nga standardizimi dhe zgjedhja e duhur e numrit të fqinjëve, ndërsa regresioni logjistik është model i thjeshtë dhe i interpretueshëm, por kufizohet në ndarje lineare.

Matricat e konfuzionit për këto modele tregojnë se ato kanë prirje të gabojnë më shpesh në klasat me pak shembuj (p.sh. klasa 0), ndërsa janë më të sakta në klasat e mëdha (klasa 4).

Rrjeti nervor MLP dhe Random Forest tregojnë një nivel më të lartë performance. MLP përfiton dukshëm nga arkitektura më e thellë (128, 64, 32) dhe funksioni i aktivizimit ReLU, që i mundëson kapjen e ndërveprimeve jolineare midis veçorive. Përdorimi i `early_stopping` bën që modeli të mos mbingarkohet, pavarësisht numrit më të madh të parametrave.

Random Forest, nga ana tjetër, arrin rezultate shumë të mira falë kombinimit të shumë pemëve dhe rregullimit përmes `max_depth` dhe `min_samples_split`, duke e bërë modelin të fortë ndaj zhurmës dhe variancës.

Grafiku i krahasimit (`classifiers_comparison.png`) e konfirmon këtë: të dy këto modele kryesojnë për sa i përket F1-macro në test, me diferenca të vogla ndaj njëri-tjetrit, ndërsa KNN dhe Regresioni Logjistik vijnë pas tyre.

Një element me rëndësi është roli i `SelectKBest` me `k=3`. Fakti që të gjithë klasifikuesit zgjedhin `k=3` si vlerë optimale tregon që një numër shumë i vogël veçorish mban informacionin kryesor për dallimin e klasave të notës, dhe se reduktimi i dimensionit nuk dëmton performancën – madje ndihmon si në shpejtësinë e trajnimit, ashtu edhe në interpretim.

Kjo vërehet edhe nga arkitektura MLP: edhe pse rrjeti ka mjaft neurone, ai punon mbi një hapësirë hyrëse më të vogël dhe më të pastruar.

Pjesa e klasterizimit ofron një pasqyrë tjetër: edhe kur nuk përdoren etiketa, algoritmet si K-Means dhe klasterizimi hierarkik arrijnë të krijojnë grupe që në një farë mase korrespondojnë me klasat e notës. Silhouette Score është vetëm mesatar, që do të thotë se ndarja nuk është perfekte, por vlerat e moderuara të ARI dhe NMI

tregojnë se struktura e dataset- it ka lidhje reale me GradeClass.

Grafikët PCA (kmeans_clusters_pca.png dhe hierarchical_clusters_pca.png)

vizualizojnë këto lidhje: disa klasterë janë qartë të veçuar, ndërsa të tjerë

mbivendosen, duke sugjeruar që faktorë të tjerë (si zhurma, ndërveprime komplekse, apo mungesa e disa veçorive relevante) e bëjnë ndarjen më të vështirë.

Në përgjithësi, projekti tregon se pipeline- i i unifikuar (parapërpunim + përzgjedhje veçorish + GridSearchCV) ofron një mënyrë sistematike për të krahasuar modele shumë të ndryshme në mënyrë të drejtë.

6. Përfundimi

Ky projekt demonstroi se si mund të ndërtohet një zinxhir i plotë analize për performancën akademike të studentëve, duke filluar nga parapërpunimi i të dhënave, përzgjedhja e veçorive dhe rregullimi i hiperparametrave, deri te krahasimi i detajuar i disa algoritmeve klasifikues dhe klasterizues.

Rezultatet treguan se modelet më të avancuara (**Random Forest dhe MLP**) arrijnë performancë shumë të mirë në parashikimin e klasës së notës GradeClass, duke arritur F1-macro dhe saktësi të larta, ndërsa modelet më të thjeshta përdoren si baselines të dobishme për krahasim.

Përdorimi i SelectKBest bëri të mundur reduktimin e numrit të veçorive pa humbur performancë, çka sugjeron se një numër i vogël indikatorësh është i mjaftueshëm për të dalluar performancën e studentëve në kategori.

Nga ana e klasterizimit, edhe pse shkalla e përputhjes me klasat reale nuk është po aq e lartë sa në klasifikim, rezultatet e K-Means dhe klasterizimit hierarkik tregojnë se dataset-i ka një strukturë të brendshme që lidhet me notat, duke hapur mundësi për analiza explorative, grupe risku, apo personalizim mbështetjeje për studentët.

Si punë e ardhshme mund të shqyrtohet përdorimi i algoritmeve të tjerë ensambël (p.sh. Gradient Boosting, XGBoost), implementimi i metodave interpretueshmërie si SHAP/LIME për Random Forest dhe MLP, si dhe trajtimi i problemit si regresion i vazhdueshëm mbi GPA në vend të klasifikimit të GradeClass.

Gjithashtu, mund të pasurohet dataset-i me veçori shtesë (p.sh. informacione historike të performancës apo faktorë socio-ekonomikë), të cilat potencialisht do të rrisnin fuqinë parashikuese të modeleve.

Figura 8 përmbledh qartë krahasimin e performancës mes katër klasifikuesve të testuar dhe konfirmon se modelet më të avancuara – Random Forest dhe Neural Network (MLP) – arrijnë vlerat më të larta si në saktësi, ashtu edhe në Precision, Recall dhe F1-macro, ndërsa KNN dhe sidomos Regresioni Logjistik mbesin prapa në të gjitha metrikat. Ky vizual tregon se për dataset-in e performancës së studentëve, kombinimi i përzgjedhjes së veçorive me modele jo-lineare dhe ensambël është më i përshtatshëm, duke mbështetur përfundimin se zgjedhja e kujdesshme e modelit është kritike për arritjen e parashikimeve të sakta në këtë problem.

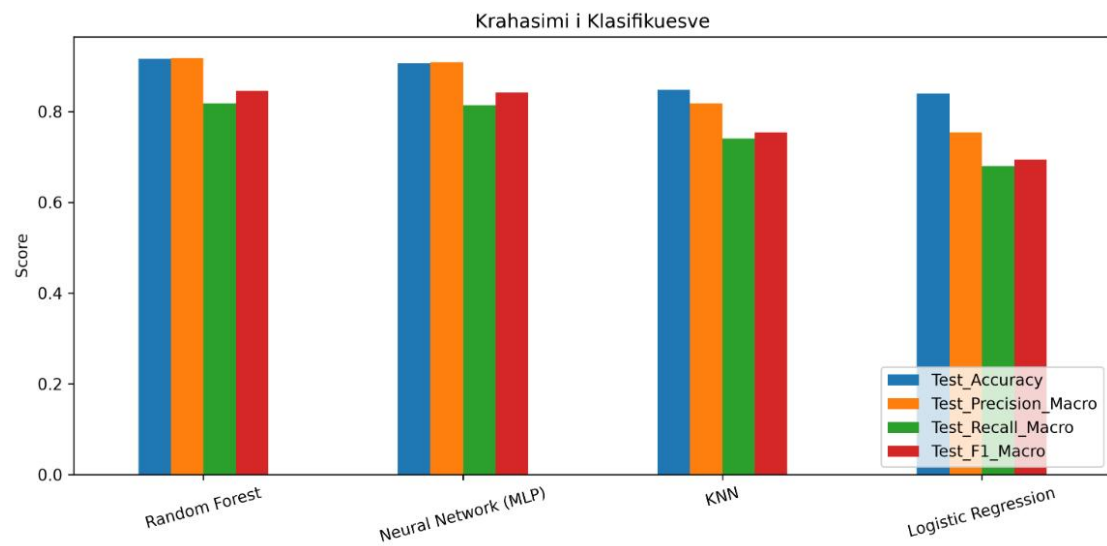


Figure 8.Krahasimi I Klasifikatoreve

7. Referencat

[1]  Students Performance Dataset  | Kaggle.

<https://www.kaggle.com/datasets/rabieelkharoua/students-performance-dataset/data>

[2] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. <https://scikit-learn.org>

[3] Fuchs, M. (2021, February 2). NN – Multi-layer perceptron classifier (MLPClassifier). Michael Fuchs Python. <https://michael-fuchs-python.netlify.app/2021/02/03/nn-multi-layer-perceptron-classifier-mlpclassifier/>